System Architecture

Technical Implementation & Deployment Guide

Chewe Technologies (pvt) LTD

Proprietary & Confidential

Generated on July 07, 2026

Chewe Technologies (pvt) LTD

1. Executive Overview

The Chewe Technologies Student Information System (SIS) is a highly scalable, real-time web application designed to revolutionize educational administration in Zimbabwe. This architecture document serves as the single source of truth for the technical deployment, database relationships, security protocols, and integration pipelines.

Our vision was to create a mobile-first, resilient application that can be utilized by executives, teachers, and students without compromising on performance or security. We achieved this through a strict separation of concerns, utilizing modern frameworks and cloud-native hosting.

Chewe Technologies (pvt) LTD

2. Technology Stack

2.1 Frontend Architecture (React + Vite)

The frontend is engineered as a Single Page Application (SPA).

- **Framework:** React 19 provides the reactive UI layer using functional components and hooks.
- **Build Tool:** Vite was selected over Create React App (CRA) due to its superior Hot Module Replacement (HMR) speeds and optimized rollup bundling for production.
- **Routing:** Pure client-side routing ensures smooth transitions without unnecessary server page reloads.
- **Styling:** Custom vanilla CSS is utilized. We eliminated heavy UI frameworks (like Bootstrap or Tailwind) to maintain absolute control over the DOM and ensure blazing fast load times on mobile devices. Data tables implement native horizontal scrolling (`overflow-x: auto`) to prevent UI breakage on small viewports.

2.2 Backend Architecture (NestJS + TypeORM)

The backend is built upon NestJS, a progressive Node.js framework that fully supports TypeScript and Object-Oriented Programming principles.

- **Modular Design:** The application is divided into strictly separated domains: AuthModule, UsersModule, AcademicsModule, DashboardModule, and AiModule.
- **Dependency Injection:** NestJS injects services directly into controllers, ensuring highly testable and decoupled code.
- **Database ORM:** TypeORM handles the translation of TypeScript classes into PostgreSQL tables, managing complex relationships (One-to-Many, Many-to-Many) cleanly.

Chewe Technologies (pvt) LTD

3. Database Schema & Relations

Data integrity is the bedrock of the Chewe System. The PostgreSQL database is structured to enforce strict relational constraints.

Core Entities:

- **User Entity:** The root authentication model. Distinguishes roles via an Enum (ADMIN, EXECUTIVE, TEACHER, STUDENT).
- **ClassRoom Entity:** Represents a physical or logical grouping of students. Students are strictly assigned to ClassRooms by Executives to prevent unauthorized self-registration.
- **Subject & Course Entities:** Subjects represent the theoretical syllabus (e.g., Mathematics). Courses represent a Subject taught by a specific Teacher within a specific ClassRoom.
- **Enrollment Entity:** The vital bridge linking a Student to a ClassRoom.
- **Assessment & Grade Entities:** Assessments define the test/exam parameters. Grades record the numerical score (0-100) linked to a specific Student and Assessment.
- **AttendanceRecord Entity:** Tracks daily roll-call statuses (Present, Absent, Late) linked to a Student and date.

Chewe Technologies (Pvt) LTD

4. External Integrations (AI & SMTP)

4.1 OpenRouter AI Engine

The system integrates deeply with the OpenRouter API to leverage Large Language Models (LLMs) for administrative efficiency. The AiService constructs secure, prompt-engineered payloads sent to OpenRouter to achieve:

- **Report Comments:** Generating dynamic, personalized end-of-term feedback for students based on their specific Z-scores and mark deficits.
- **Executive Summaries:** Translating raw database KPI metrics (like school-wide attendance averages) into natural language briefings.

4.2 Nodemailer SMTP

Email operations are handled via Nodemailer. By utilizing a Google App Password attached to `contact.chewetechnologies@gmail.com`, the system can securely dispatch JWT-signed Password Reset links directly to user inboxes without exposing core credentials.

Chewe Technologies (pvt) LTD

5. Server Deployment & Security

The application is hosted on an enterprise-grade Ubuntu 24.04 VPS provided by Contabo. The deployment pipeline is streamlined for maximum uptime.

Process Management (PM2)

Both the Vite frontend (served via local HTTP) and the NestJS backend are daemonized using PM2. This ensures that if the Node runtime encounters a fatal exception, the process is instantly restarted, minimizing downtime.

```
pm2 start npm --name "chewe-frontend" -- run preview
pm2 start dist/main.js --name "chewe-backend"
```

Nginx Reverse Proxy & SSL

Nginx acts as the gatekeeper. It listens on port 443 (HTTPS), terminating the SSL connection managed by Let's Encrypt Certbot. It then proxies static frontend requests to the Vite process, and dynamically routes any request matching `/api/*` to the NestJS backend on port 5000.

Security Protocols

- **JWT Authentication:** Stateless token-based auth ensures scalability. Tokens are signed with a robust secret key.
- **Bcrypt Hashing:** Passwords are never stored in plaintext. They undergo multiple rounds of bcrypt salting before database insertion.
- **Firewall (UFW):** Only ports 80 (HTTP), 443 (HTTPS), and 22 (SSH) are exposed to the public internet. Internal database ports (5432) are strictly walled off.

Chewe Technologies (pvt) LTD